Updating Kogger Sonar Firmware Without KoggerApp

Host-side specification for third-party integrators

Every statement in this document is tagged with where it comes from:

Tag	Meaning
`[code]`	read from the KoggerApp sources
`[measured]`	observed in a trace of a real firmware update, 18 Aug 2026
`[unverified]`	present in the code, but never executed during that trace

References of the form `src/file.h:123` point into KoggerApp, which is open source, and can be checked directly.

> **Scope — and one hard prohibition**
>
> This document describes **only what the host does**.
>
> **The `.ufw` encryption method and the internal design of the firmware must not be disclosed under any circumstances** — not in this document, not in correspondence, not in answers to follow-up questions. This is a prohibition, not a note about scope.
>
> Everything described below is open: the exchange protocol lives in the KoggerApp sources and is freely available.

---

1. Summary

What has to be implemented, in one paragraph:

> Open the `.ufw` file as an opaque binary blob. Send the command to enter the bootloader. Cut the file into 96-byte chunks and send them in order, numbered from 1, waiting for an answer to each. When the data runs out, send the command to start the firmware.

The host does not parse the firmware file. `[code]` `IDBinUpdate::setUpdate` (`src/id_binnary.cpp:1120`) takes an array of bytes and passes it through unchanged. Integrity and authenticity are verified entirely on the device.

The practical consequence: **you do not need to know the `.ufw` format** in order to implement this.

---

2. Protocol messages

`[code]` `src/proto_binnary.h:78-83`

ID	Name	Role
`0x20`	`ID_VERSION`	confirm the link before starting (optional)
`0x24`	`ID_BOOT`	enter the bootloader (v0) and leave it (v1)
`0x25`	`ID_UPDATE`	carry the firmware body

A frame is built as `(type, version, ID, device address)`. The type is `SETTING` for all three.

Device address is `0x00`. At this stage of integration a single device on the line, answering at address zero, is assumed. Addressing several devices on one line is out of scope here.

2.1 `ID_BOOT` — bootloader control

`[code]` `src/id_binnary.cpp:1094` (enter), `:1102` (leave)

Version	Action	Payload
`v0`	enter the bootloader (device reboots)	`U4` confirmation word
`v1`	leave it and start the firmware	`U4` confirmation word

One message, two versions — do not confuse them.

Confirmation word: `0xC96B5D4A`, the constant `IDBin::m_key` (`src/id_binnary.h:109`), sent as `U4`. If this document and the sources ever disagree, believe the sources — that is where the value is maintained.

2.2 `ID_UPDATE` — firmware chunk (host → device)

`[code]` `IDBinUpdate::putUpdate`, `src/id_binnary.cpp:1131`

Offset	Type	Field
0	`U2`	packet number, starting at **1**
2	`U1[]`	body, up to **96** bytes

Chunk size: `_packetSize = 32*3` (`src/id_binnary.h:1004`). The final chunk is shorter.

The host keeps **two independent counters**: the packet number and the offset into the file. When they drift apart, the device reports both.

2.3 `ID_UPDATE` — device answer, form A: progress structure

`[code]` `ID_UPGRADE_V0`, `src/id_binnary.h:955`. Packed, no padding.

Offset	Type	Field	Meaning
0	`U2`	`lastNumMsg`	last packet number accepted
2	`U4`	`lastOffset`	offset the device expects to continue from
6	`U1`	`type`	status code, below
7	`U1`	`rcvNumMsg`	

Response to `type` — `[code]` `src/device/dev_driver.cpp:1534`:

`type`	Meaning	Host action
`0`	accepted	send the next chunk
`1`	drift, backwards	log it and carry on
`2`	drift, forwards	**move to the device's `lastNumMsg` / `lastOffset`** and continue from there
`> 2`	fatal error	abort

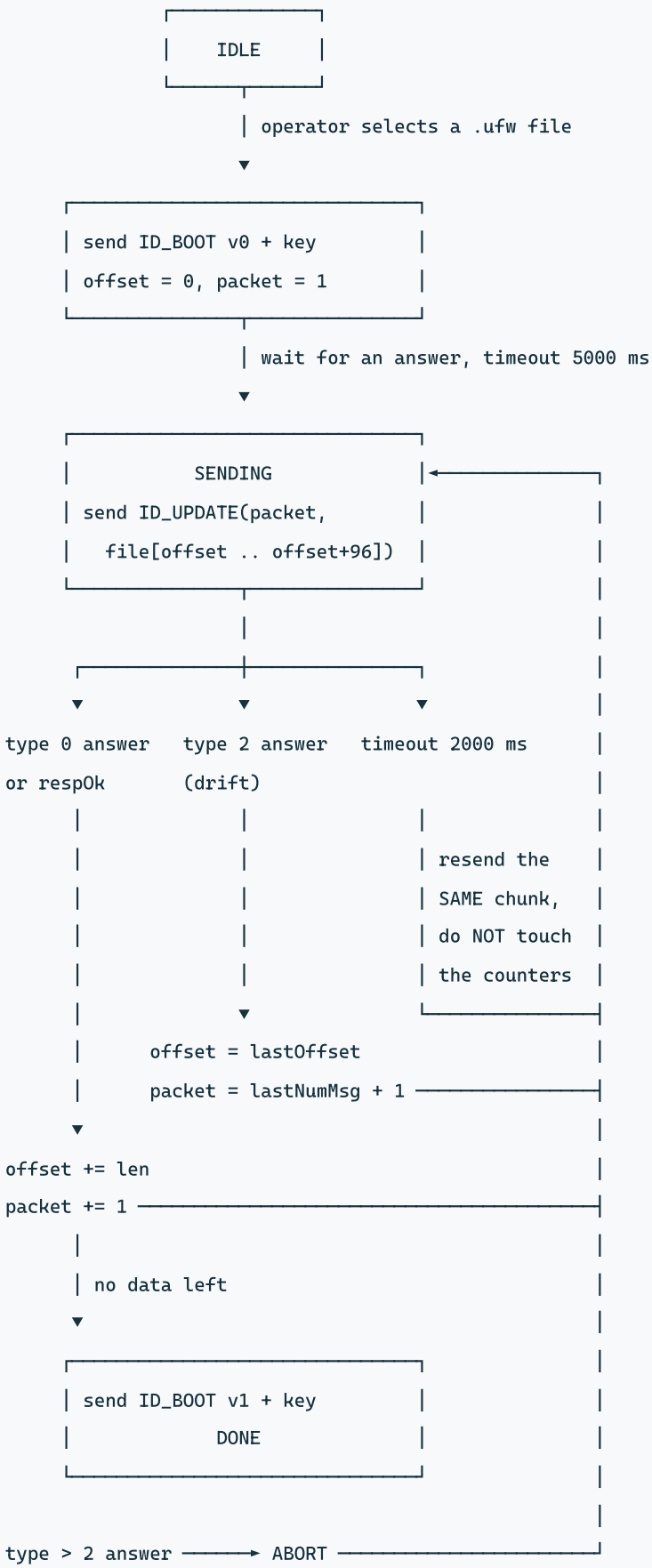
2.4 `ID_UPDATE` — device answer, form B: plain acknowledgement

`[code]` `src/device/dev_driver.cpp:1586`. The device answers with an ordinary `respOk` and no structure. The host sends the next chunk.

An implementation must handle both forms. KoggerApp starts out assuming form B and switches to form A on the first well-formed version-0 answer.

> `[measured]` On the device tested, **every** answer was form B. See §6.

3. State machine



4. Pseudocode

```
function update_firmware(file_bytes, device_address = 0x00):
    send(ID_BOOT, version = 0, payload = KEY)      # enter the bootloader
    offset  = 0
    packet  = 1
    timeout = 5000 ms                              # covers the reboot

    while offset < len(file_bytes):
        chunk = file_bytes[offset : offset + 96]
        send(ID_UPDATE, version = 0, payload = u16(packet) + chunk)

        answer = wait_for_answer(timeout)

        if answer is TIMEOUT:
            continue                               # resend; counters untouched

        timeout = 2000 ms                          # after the first answer

        if answer is PLAIN_OK:                     # form B
            offset += len(chunk)
            packet += 1
            continue

        if answer.type == 0:                       # form A, all well
            offset += len(chunk)
            packet += 1
        elif answer.type == 1:
            log(answer); offset += len(chunk); packet += 1
        elif answer.type == 2:                     # the device dictates position
            offset = answer.lastOffset
            packet = answer.lastNumMsg + 1
        else:
            abort("fatal error")

    send(ID_BOOT, version = 1, payload = KEY)      # start the firmware
```

5. Three traps

5.1 A retransmit must not advance the counters

`[code]` `src/device/dev_driver.cpp:1802`. On timeout the **same** chunk is sent again: `putUpdate(false)`, where `false` disables the automatic advance.

An implementation that moves on after a timeout will corrupt the firmware silently.

5.2 Two timeouts, and no hard-coded delay

`[code]` 5000 ms before the first answer, 2000 ms afterwards. The first covers the reboot into the bootloader, which `[measured]` took **1732 ms**.

That figure is not guaranteed and may differ between devices. A correct implementation therefore does not wait a fixed period: it sends the first chunk and **waits for an answer on the timeout, resending as needed**. This is exactly what KoggerApp does, and why its first timeout is three times the delay actually observed.

5.3 On drift, the device outranks the host

On `type == 2` the host must move to the position the device reported, rather than insisting on its own counter.

---

6. Measured exchange

`[measured]` Trace of a real update, 18 Aug 2026, over USB-UART (CP2105).

Quantity	Value
Firmware size	43,900 bytes
Body packets	458 × 96 bytes
Entering the bootloader	**1,732 ms**
Body transfer	4,370 ms
Total	6,102 ms
Throughput	≈ 9.8 kB/s
Per-packet interval	9.6 ms mean (3…79)
Retransmits	0
Drift events	0

What the trace confirmed

96-byte chunking, numbering from 1, and the closing `ID_BOOT v1` all behave as the code describes. The reboot took ≈ 1.7 s, so the 5000 ms first timeout has roughly threefold headroom.

What the trace disproved in the first draft of this document

The progress-structure branch never executed. The device answered plain `respOk` to all 457 replies. Section 2.3 and the `type` handling are described **from the code only** and remain `[unverified]`.

The consequence for implementers: **form B is mandatory, form A is desirable.** Start with the plain acknowledgement — you will certainly meet it.

A numbering quirk visible in logs

`[measured]` On the third packet the host sends **two chunks back to back** `[code]` (`src/device/dev_driver.cpp:1505`). In the trace the numbers run `2, 4, 5, 6…`.

Reproducing this is not required. But when comparing your exchange against KoggerApp's, a one-packet discrepancy at the start is this, not a fault.

---

7. Suggested order of work

1. Transport and framing for the binary protocol — shared with all other communication with the device, not specific to updating.
2. `ID_VERSION` — confirm the link works.
3. `ID_BOOT v0` → wait → `ID_UPDATE` loop → `ID_BOOT v1`.
4. Timeout retransmit, and support for the plain `respOk` answer.
5. Handling of `type` 1 and 2 — once a device that sends them is available.

Step 4 is not optional. Without it, updates succeed only on a perfect link.

---

8. Failure behaviour and responsibility

8.1 Link or power lost mid-transfer

The device stays in the bootloader, waiting for the next valid frame.

Consequences:

- There is **no rollback** to the previous firmware on timeout. A device with an interrupted update remains in the bootloader until it is flashed again.
- Recovery needs nothing special: keep sending valid frames. Restarting from the first packet is acceptable.
- It follows that after a dropped link you should **not re-send** `ID_BOOT v0` — the device is already in the bootloader.

8.2 Matching firmware to device

The bootloader checks this.

The host checks nothing `[code]` — it transmits the bytes it was given. What happens if the wrong firmware is sent is decided entirely on the device, and no host implementation can influence it.

Even so, keeping your own record of which file belongs to which model is worth doing: a rejection at the end of a transfer is cheaper than no rejection, and more expensive than not starting.

8.3 Devices that send the progress structure

Whether any exist in the fleet is not currently known.

The device tested has none — every answer was a plain `respOk`, so §2.3 stays `[unverified]`.

The recommendation is unchanged: **begin with form B**. Add form A when a device that exercises it is available to test against.

Appendix: capturing a trace yourself

KoggerApp contains per-packet update diagnostics, disabled by default. Enable them with an environment variable:

```
set QT_ASSUME_STDERR_HAS_CONSOLE=1
set KOGGER_FW_TRACE=1
KoggerApp.exe > fwtrace.txt 2>&1
```

Events written: `start`, `out` (packet, offset, remaining), `in` (type, device counters against host counters), `retransmit`, `runFW` — all with millisecond timestamps.

Known gap: **the first packet does not appear in the trace.** It is sent from a timer, bypassing the instrumented function. This does not affect analysis — the numbering and offsets still reconcile.